Universidade do Minho
Escola de Engenharia

Departamento de Informática

Licenciatura em Engenharia Informática

Computação Gráfica

Trabalho Prático (Fase 2)

Grupo n.º 28

Carlos Daniel Lopes Cunha, nº A106910

Pedro Nuno de Bastos Pinho Costa, nº A107375

Francisco Queirós Ribeiro da Costa Maia, nº A108962

Índice

1 Introdução **1**

2 Arquitetura da Scene Graph **2**
2.1 Estrutura de Dados 2
2.2 Leitura do Ficheiro XML 2

3 Transformações Geométricas **3**
3.1 Translação 3
3.2 Escala 3
3.3 Rotação 3
3.4 Composição e Ordem das Transformações 4

4 Herança de Transformações **4**
4.1 Mecanismo de *Push/Pop* 4
4.2 Implementação 4
4.3 Validação de Resultados 5

5 Formato XML de Configuração **7**
5.1 Câmara 7
5.2 Grupos e Transformações 7
5.3 Modelos 7

6 Sistema Solar **8**
6.1 Hierarquia 8
6.2 Sistema Terra-Lua 8
6.3 Escalabilidade e Modelos 9

7 Extras **10**
7.1 Câmara Interativa 10

8 Conclusão **10**

1 Introdução

O presente relatório descreve o trabalho desenvolvido na segunda fase do trabalho prático da unidade curricular de Computação Gráfica.

O objetivo desta fase consistiu em estender o motor gráfico desenvolvido na Fase 1, acrescentando suporte para **transformações geométricas hierárquicas** através de uma *scene graph*. Enquanto que na fase anterior o motor se limitava a carregar e renderizar modelos 3D de forma plana, nesta fase é possível organizar os modelos em grupos aninhados, onde cada grupo pode ter associadas transformações geométricas próprias que são herdadas pelos seus subgrupos.

As principais adições desta fase foram:

- Implementação de uma *scene graph* baseada numa estrutura de árvore de grupos, suportando aninhamento arbitrário;
- Suporte para as três transformações geométricas fundamentais: translação, rotação e escala;
- Herança de transformações entre nós pai e nós filho da hierarquia;
- Extensão do formato XML de configuração para descrever cenas hierárquicas;
- Construção de uma cena demonstrativa estática do sistema solar, modelando o Sol, os oito planetas, a Lua da Terra e os anéis de Saturno.

O gerador de primitivas desenvolvido na Fase 1 não sofreu alterações nesta fase, uma vez que os requisitos se centraram exclusivamente no motor de renderização e na organização hierárquica da cena.

2 Arquitetura da Scene Graph

Para suportar cenas hierárquicas com transformações geométricas, foi adotada uma *scene graph* baseada numa estrutura de árvore de grupos. Esta abordagem permite organizar os modelos 3D de forma hierárquica, onde cada nó da árvore representa um grupo que pode conter transformações, modelos e subgrupos.

2.1 Estrutura de Dados

Cada nó da árvore é representado por um **grupo**, que encapsula três componentes principais:

- **Lista de transformações** — sequência ordenada de transformações geométricas (translação, rotação e escala) a aplicar ao grupo;
- **Lista de modelos** — conjunto de modelos 3D associados ao grupo, referenciados pelos seus ficheiros `.3d`;
- **Lista de filhos** — subgrupos aninhados, que herdam as transformações do grupo pai.

2.2 Leitura do Ficheiro XML

A cena é carregada uma única vez no arranque do motor, a partir de um ficheiro XML de configuração. O processo de leitura percorre recursivamente os elementos `<group>` do ficheiro, populando a estrutura em árvore descrita acima. Para cada grupo encontrado, são lidos por ordem:

1. As transformações contidas na tag `<transform>`;
2. Os modelos listados na tag `<models>`, cada um carregado do respetivo ficheiro `.3d` para memória;
3. Os subgrupos `<group>` filhos, processados recursivamente.

Esta abordagem garante que toda a informação da cena fica armazenada em memória antes do início da renderização, evitando leituras de ficheiro durante o ciclo de desenho. A estrutura recursiva de grupos foi escolhida por três razões principais. Em primeiro lugar, reflete diretamente a hierarquia descrita no ficheiro XML, tornando o mapeamento entre ficheiro e estrutura em memória natural e simples. Em segundo lugar, permite expressar relações de dependência espacial entre objetos, por exemplo, a lua como filha da Terra. Por fim, a recursividade da estrutura adapta-se a qualquer profundidade de aninhamento sem necessidade de lógica adicional, sendo que a renderização segue o mesmo padrão recursivo da leitura.

3 Transformações Geométricas

As transformações geométricas suportadas nesta fase são a translação, a rotação e a "scale". Cada uma é representada internamente por uma matriz 4×4 em coordenadas homogéneas, o que permite combinar transformações por multiplicação matricial e aplicá-las a pontos 3D de forma uniforme.

3.1 Translação

A translação desloca um objeto no espaço de acordo com um vetor (t_x, t_y, t_z). A sua matriz é:

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{1}$$

3.2 Escala

A escala altera as dimensões de um objeto em cada eixo de forma independente, de acordo com os fatores (s_x, s_y, s_z):

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{2}$$

3.3 Rotação

A rotação é definida por um ângulo θ e um eixo arbitrário (a_x, a_y, a_z). A título ilustrativo, apresenta-se a matriz de rotação em torno do eixo Y:

$$R_Y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{3}$$

Para um eixo arbitrário, a rotação é obtida pela fórmula de *Rodrigues*, que o OpenGL implementa internamente através da função `glRotatef(angle, ax, ay, az)`.

3.4 Composição e Ordem das Transformações

Quando um grupo contém múltiplas transformações, a matriz final aplicada a cada vértice é o produto das matrizes individuais:

$$M = T \cdot R \cdot S \tag{4}$$

A multiplicação de matrizes não é comutativa, pelo que a ordem das transformações é relevante e é respeitada conforme a sequência definida no ficheiro XML. As transformações são aplicadas da direita para a esquerda, ou seja, primeiro a escala, depois a rotação e por fim a translação. Esta convenção é a mais comum em computação gráfica, pois produz resultados intuitivos: o objeto é primeiro dimensionado, depois orientado e por fim posicionado no espaço.

4 Herança de Transformações

Um dos requisitos centrais desta fase é que as transformações de um grupo pai sejam herdadas por todos os seus subgrupos. Isto significa que, ao aplicar uma translação a um grupo, todos os modelos e grupos filhos são deslocados de forma solidária, sem necessidade de repetir as transformações em cada nó filho.

4.1 Mecanismo de *Push/Pop*

Este comportamento é obtido através da pilha de matrizes do OpenGL, usando as funções `glPushMatrix()` e `glPopMatrix()`. No início da renderização de cada grupo, o estado atual da matriz de transformação é guardado na pilha. As transformações do grupo são então aplicadas e acumuladas sobre esse estado. Ao terminar a renderização do grupo e de todos os seus filhos, o estado original é restaurado, garantindo que os grupos irmãos não são afetados.

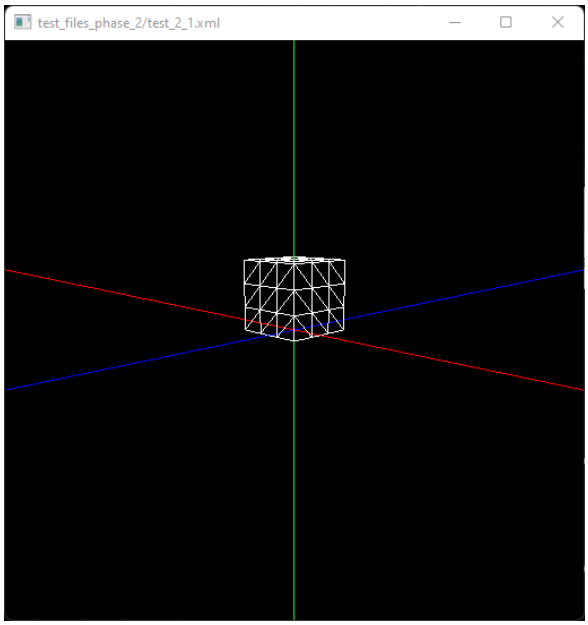
4.2 Implementação

A renderização da *scene graph* é feita de forma recursiva. Para cada grupo, o motor guarda o estado da matriz, aplica as transformações, desenha os modelos e visita recursivamente os filhos.

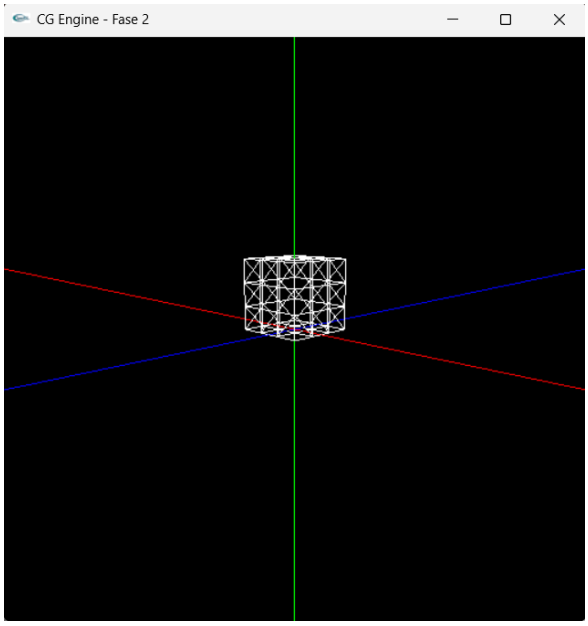
A chamada a `glPushMatrix()` no início e a `glPopMatrix()` no final garantem que as transformações aplicadas num grupo não se propagam para grupos irmãos nem para o nível superior da hierarquia, apenas descem para os filhos, que são visitados enquanto a matriz acumulada ainda está ativa.

4.3 Validação de Resultados

De forma a validar a correta implementação da herança de transformações, estas são as comparações entre os resultados esperados fornecidos pelos docentes e os obtidos pelo motor desenvolvido.

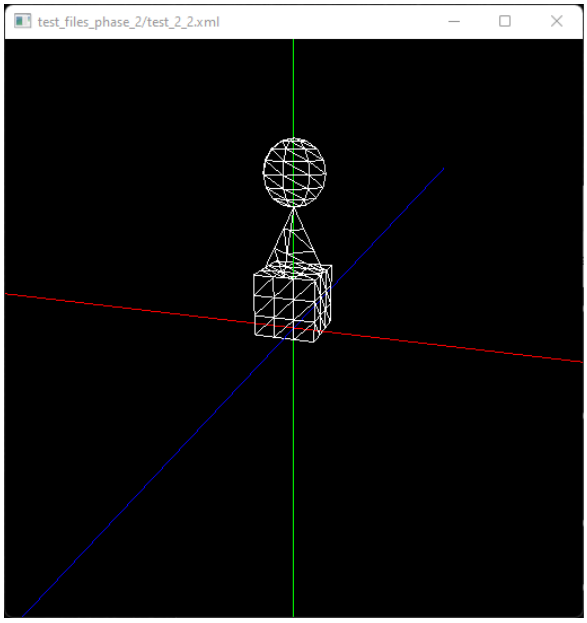


Resultado esperado

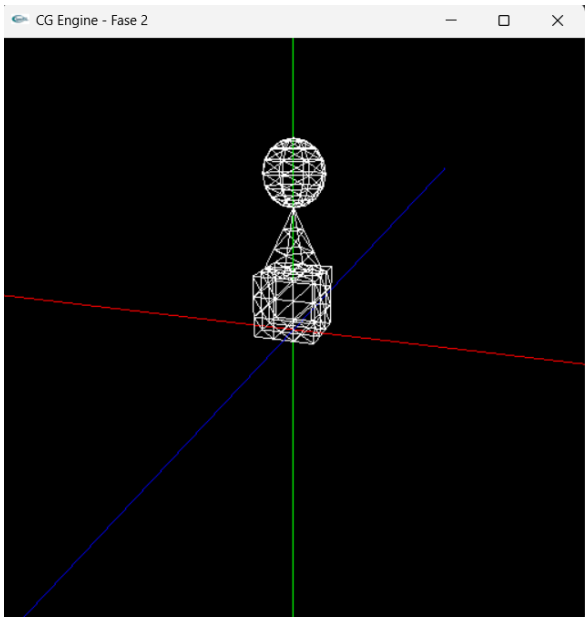


Resultado obtido

Teste 1

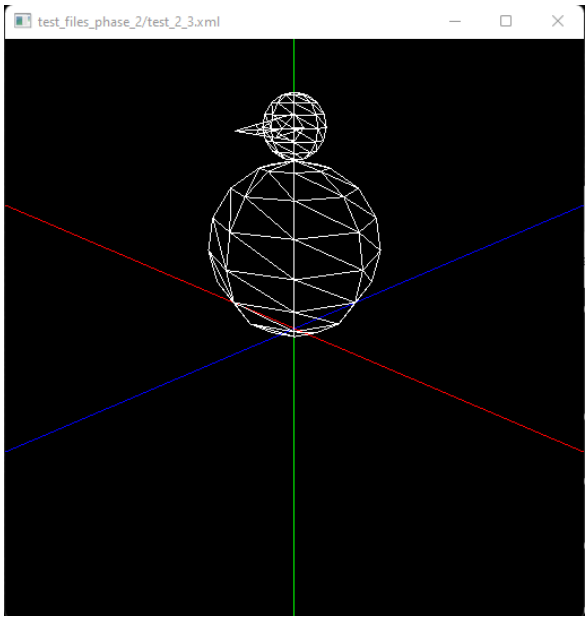


Resultado esperado

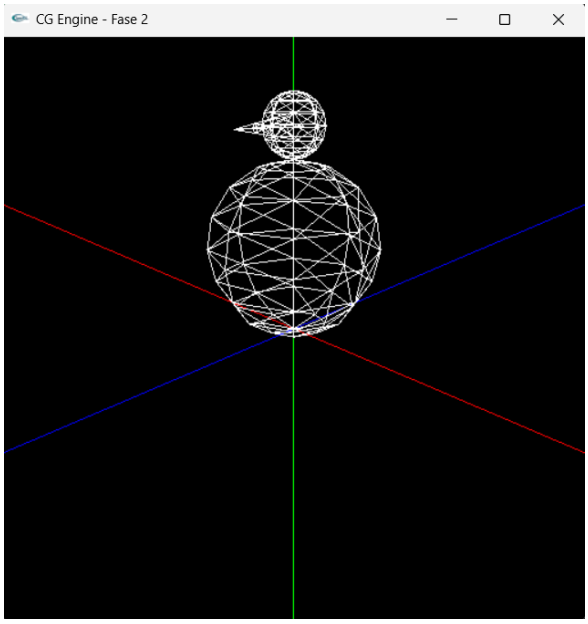


Resultado obtido

Teste 2

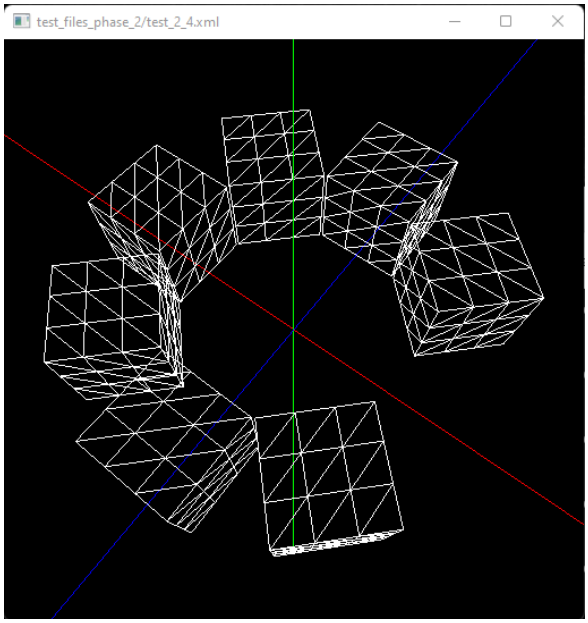


Resultado esperado

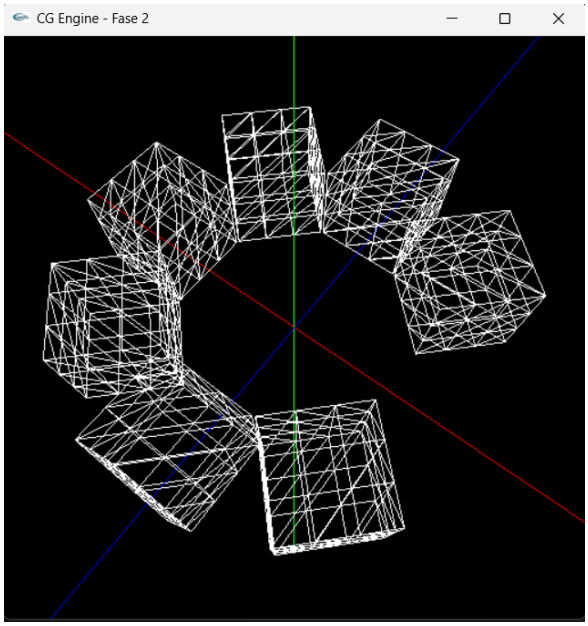


Resultado obtido

Teste 3



Resultado esperado



Resultado obtido

Teste 4

5 Formato XML de Configuração

O ficheiro XML de configuração é o ponto de entrada do motor. Define a janela, a câmara e a hierarquia de grupos que compõem a cena. A sua estrutura foi desenhada para ser simples, legível e diretamente mapeável para a *scene graph* em memória. O elemento raiz `<world>` contém três secções principais: `<window>`, `<camera>` e `<group>`.

5.1 Câmara

A câmara é definida por quatro sub-elementos: `<position>`, `<lookAt>`, `<up>` e `<projection>`. Os três primeiros definem a posição, o ponto de foco e o vetor up da câmara, respetivamente. O elemento `<projection>` define o campo de visão (`fov`), o plano de corte próximo (`near`) e o plano de corte distante (`far`).

5.2 Grupos e Transformações

Cada `<group>` pode conter, por esta ordem: um bloco `<transform>` com as transformações geométricas, um bloco `<models>` com os modelos a renderizar, e subgrupos `<group>` filhos. O seguinte excerto exemplifica um grupo com as três transformações e um modelo associado:

```
<group>
    <transform>
        <translate x="4" y="0" z="0" />
        <rotate angle="30" x="0" y="1" z="0" />
        <scale x="2" y="0.3" z="1" />
    </transform>
    <models>
        <model file="cone.3d" />
    </models>
</group>
```

A decisão de encapsular as transformações dentro de uma tag `<transform>` explícita, em vez de as colocar diretamente no grupo, tem dois objetivos: tornar a estrutura mais legível e facilitar a distinção semântica entre transformações, modelos e subgrupos durante o processo de leitura.

A ordem das tags `<translate>`, `<rotate>` e `<scale>` dentro de `<transform>` é relevante, uma vez que determina a ordem de multiplicação das matrizes. O motor respeita e aplica as transformações exatamente pela ordem em que aparecem no ficheiro.

5.3 Modelos

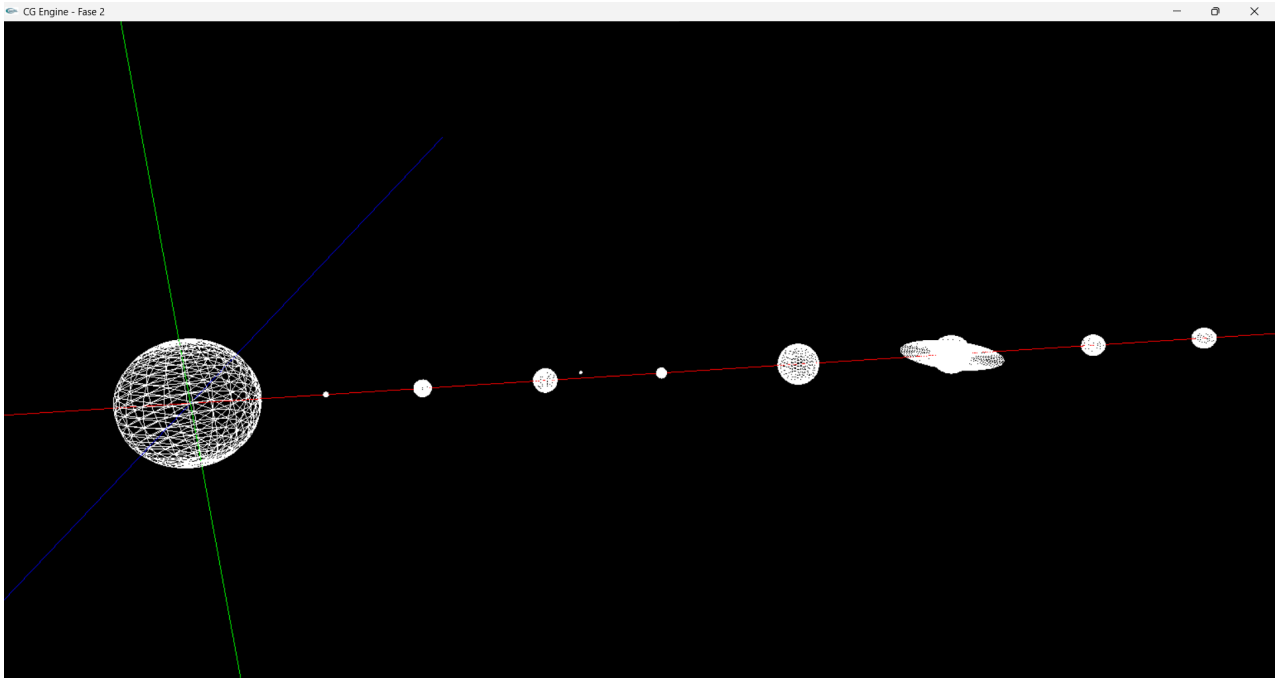
Cada modelo é referenciado pelo atributo `file` do elemento `<model>`, que indica o caminho para o ficheiro `.3d` gerado pela aplicação geradora. O motor carrega todos os modelos para memória no arranque, evitando leituras de disco durante o ciclo de renderização.

6 Sistema Solar

Para demonstrar as capacidades do motor gráfico e a flexibilidade da *scene graph*, foi desenvolvida uma cena representativa do Sistema Solar. Esta demonstração foca-se na organização hierárquica e na aplicação de transformações sucessivas para modelar relações espaciais entre corpos celestes.

6.1 Hierarquia

A "scene" foi estruturada seguindo a lógica de dependência gravitacional. O Sol posiciona-se na origem do sistema de coordenadas , servindo como nó pai de todos os planetas. Cada planeta, por sua vez, define o seu próprio sistema de coordenadas local, onde podem existir subgrupos para luas ou anéis.



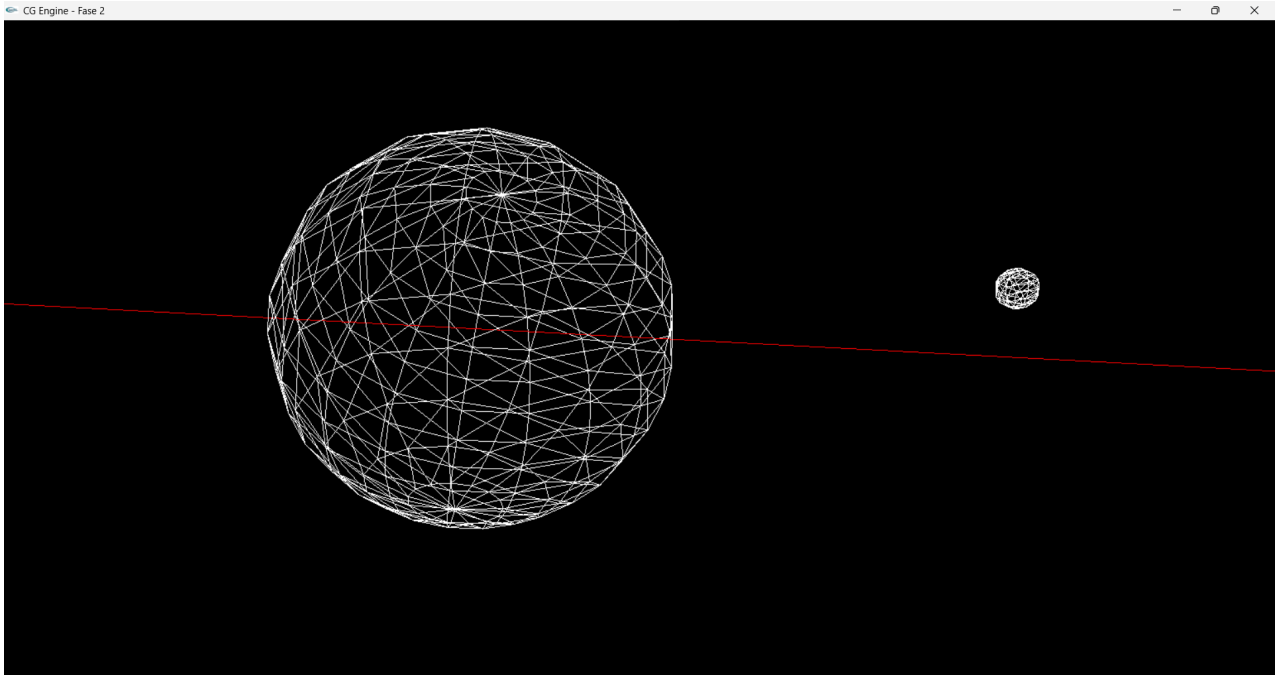
Renderização do Sistema Solar no motor gráfico.

6.2 Sistema Terra-Lua

O sistema Terra-Lua exemplifica a utilidade da herança de transformações. No XML, a Lua é definida como um subgrupo da Terra. Isto implica que qualquer translação aplicada ao grupo da Terra (para a posicionar na sua órbita em relação ao Sol) é automaticamente herdada pela Lua.

- **Grupo Terra:** Aplica uma translação $T(d_{sol}, 0, 0)$ para posicionar o planeta.
- **Grupo Lua (Filho):** Aplica uma translação adicional $T(d_{terra}, 0, 0)$.

O resultado final na matriz de visualização para a Lua é o produto $M = T_{terra} \cdot T_{lua}$, garantindo que a Lua se mantém a uma distância constante da Terra, independentemente da posição desta última no espaço global.

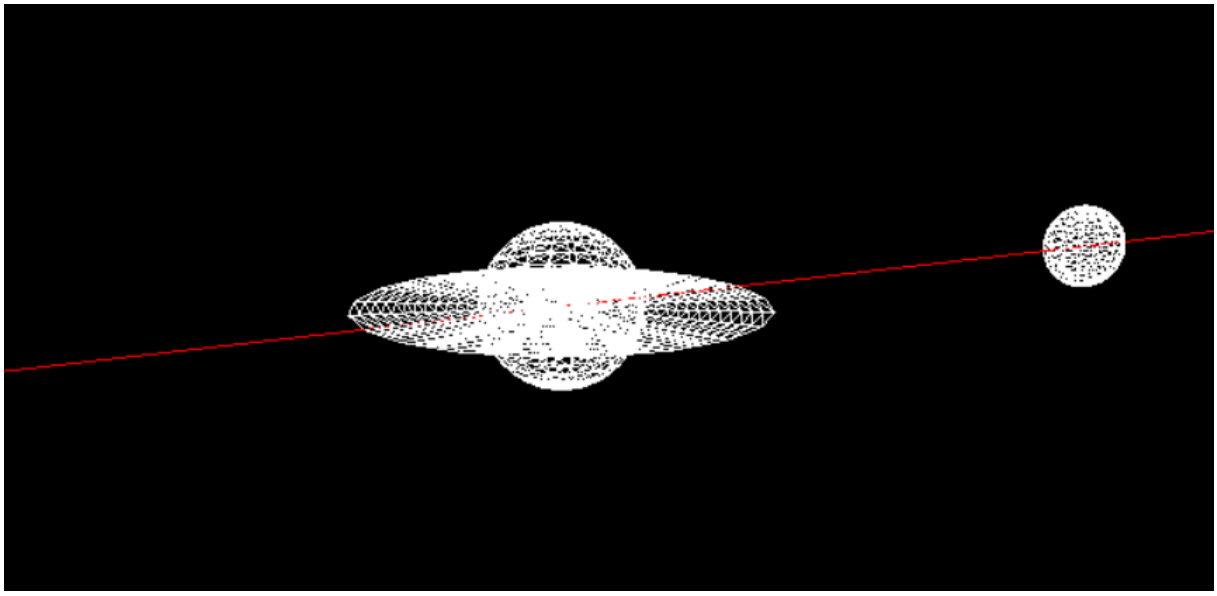


Terra e Lua.

6.3 Escalabilidade e Modelos

Para conferir realismo à cena, foram utilizados diferentes fatores de escala e resoluções de malha. Os planetas gigantes, como Júpiter e Saturno, utilizam esferas com maior densidade de triângulos para evitar artefactos visuais, enquanto planetas menores utilizam malhas mais simples para otimizar a renderização.

Os anéis de Saturno foram modelados como um subgrupo contendo um modelo de disco achatado (*ring*), escalado drasticamente no eixo Y para simular a sua espessura mínima, demonstrando como a transformação `scale` pode reutilizar primitivas básicas para criar formas complexas.



Saturno.

7 Extras

Para além dos requisitos base da Fase 2, foram implementadas funcionalidades adicionais que melhoram a usabilidade do motor e a eficiência do sistema.

7.1 Câmara Interativa

Embora o ficheiro XML defina os parâmetros iniciais da câmara, o motor permite a exploração dinâmica da cena através do teclado e rato. Esta funcionalidade utiliza coordenadas esféricas para permitir uma navegação intuitiva:

- **Rotação (Rato/Teclado):** O utilizador pode orbitar o ponto de foco (*lookAt*) alterando os ângulos de *alpha* e *beta*.
- **Zoom (Teclado):** As teclas de direção (cima/baixo) ou teclas específicas permitem aproximar ou afastar a câmara da origem, alterando o raio da esfera de visualização.

Esta interatividade é processada no ciclo de eventos do GLUT, atualizando a posição da câmara antes de cada chamada à função `gluLookAt`.

8 Conclusão

A segunda fase do projeto permitiu a transição bem-sucedida de uma renderização estática para um sistema de *scene graph* flexível. A organização hierárquica, suportada pela pilha de matrizes do OpenGL (*push/pop*), provou ser uma solução robusta para modelar relações espaciais complexas, como sistemas planetários, de forma eficiente.

Em suma, os objetivos foram atingidos: o motor agora processa ficheiros XML com aninhamento , gere o carregamento de modelos em memória de forma otimizada e oferece interatividade através de uma câmara exploradora. O sistema encontra-se sólido e preparado para a integração de modelos de iluminação e movimento nas fases seguintes.